Akansha Balhara

- Roll no.: 1/23/SET/BCS/433
- Date : 24/02/2026
- Exp no. : 6

Email spam/ham classifiaction(MDP)

```
# ============================
# PROGRAM 1 — TRAIN/TEST = 80/20
# ============================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix


# ============================
# 1. LOAD DATA
# ============================
df = pd.read_csv("/content/spam_or_not_spam.csv")
df = df[['email', 'label']].dropna().reset_index(drop=True)

X_full = df["email"]
y_full = df["label"].values.reshape(-1, 1)

vectorizer = TfidfVectorizer(stop_words="english", max_features=4000,
ngram_range=(1,2))
vectorizer.fit(X_full)


# ============================
# 2. ACTIVATION FUNCTIONS
# ============================
def sigmoid(z): return 1 / (1 + np.exp(-z))
def relu(z): return np.maximum(0, z)
def relu_derivative(z): return (z > 0).astype(float)
def bce_loss(y, y_pred): return -np.mean(y*np.log(y_pred + 1e-8) + (1-
y)*np.log(1-y_pred + 1e-8))


# ============================
# 3. NEURAL NETWORK FUNCTION
# ============================
```

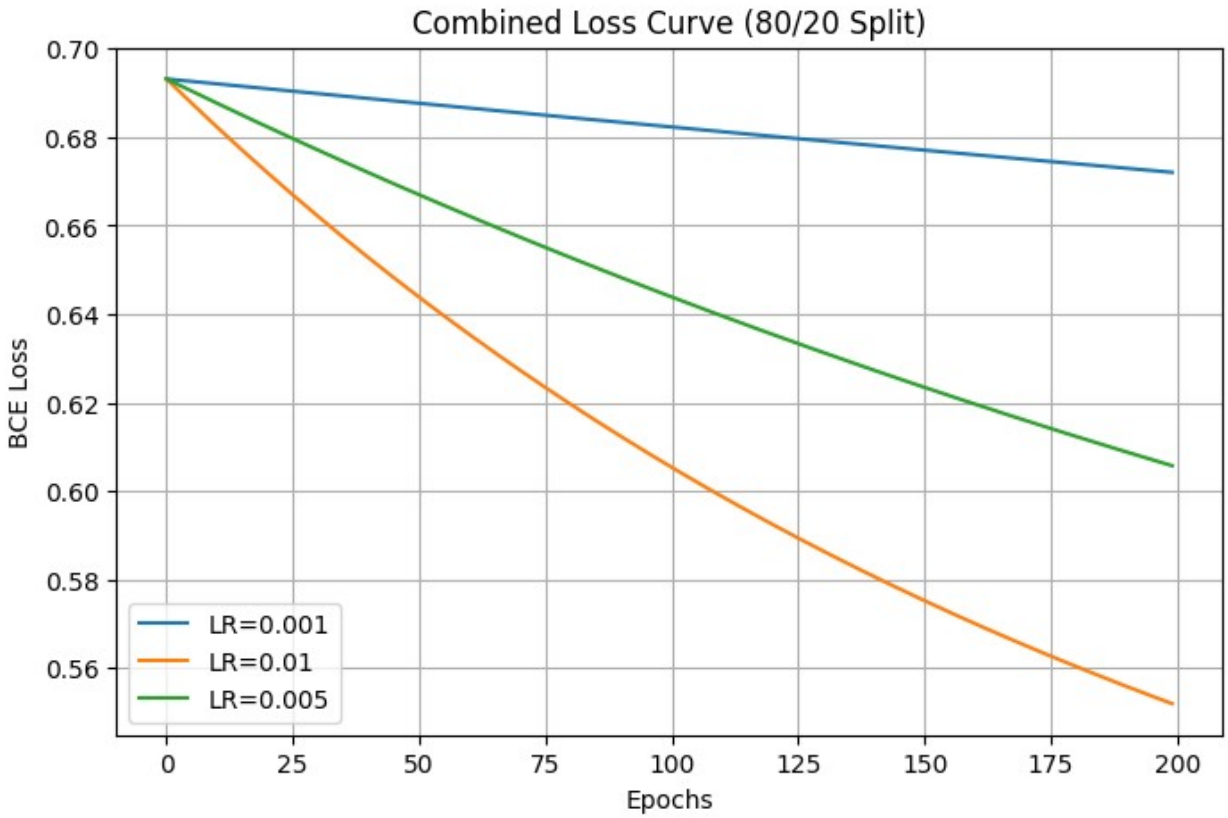
```python
def train_model(X_train_vec, y_train, X_test_vec, y_test, lr):

    epochs = 200
    m, n = X_train_vec.shape

    h1, h2 = 64, 32   # two hidden layers

    # Random initialization
    W1 = np.random.randn(n, h1) * 0.01
    b1 = np.zeros((1, h1))

    W2 = np.random.randn(h1, h2) * 0.01
    b2 = np.zeros((1, h2))

    W3 = np.random.randn(h2, 1) * 0.01
    b3 = np.zeros((1, 1))

    loss_history = []
    acc_history = []

    for epoch in range(epochs):

        # Forward pass
        Z1 = np.dot(X_train_vec, W1) + b1
        A1 = relu(Z1)

        Z2 = np.dot(A1, W2) + b2
        A2 = relu(Z2)

        Z3 = np.dot(A2, W3) + b3
        A3 = sigmoid(Z3)

        loss = bce_loss(y_train, A3)
        loss_history.append(loss)

        train_pred = (A3 >= 0.5).astype(int)
        acc = accuracy_score(y_train, train_pred)
        acc_history.append(acc)

        # Backpropagation
        dZ3 = A3 - y_train
        dW3 = np.dot(A2.T, dZ3) / m
        db3 = np.mean(dZ3, keepdims=True)

        dA2 = np.dot(dZ3, W3.T)
        dZ2 = dA2 * relu_derivative(Z2)
        dW2 = np.dot(A1.T, dZ2) / m
        db2 = np.mean(dZ2, keepdims=True)

        dA1 = np.dot(dZ2, W2.T)
        dZ1 = dA1 * relu_derivative(Z1)
```

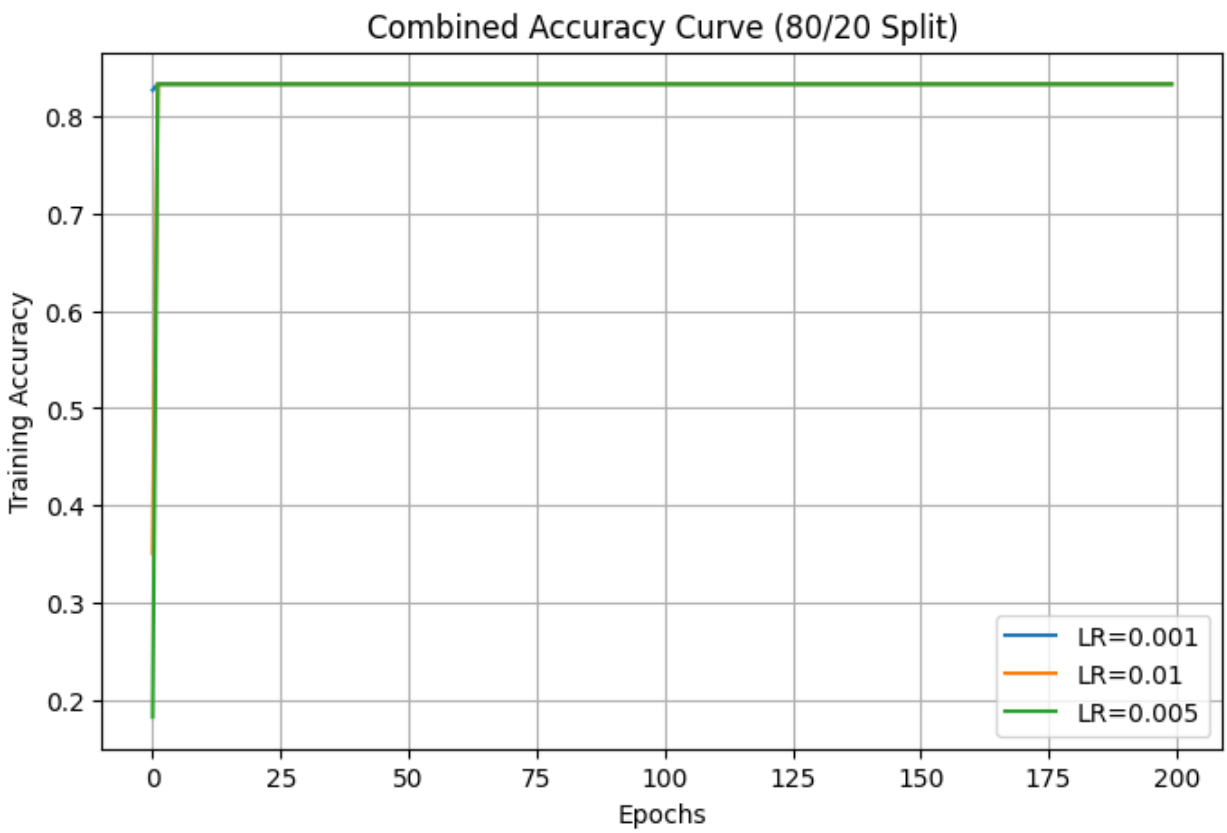
```
        dW1 = np.dot(X_train_vec.T, dZ1) / m
        db1 = np.mean(dZ1, keepdims=True)

        # Gradient update
        W1 -= lr * dW1
        b1 -= lr * db1
        W2 -= lr * dW2
        b2 -= lr * db2
        W3 -= lr * dW3
        b3 -= lr * db3

    # Evaluation
    Z1 = np.dot(X_test_vec, W1) + b1
    A1 = relu(Z1)

    Z2 = np.dot(A1, W2) + b2
    A2 = relu(Z2)

    Z3 = np.dot(A2, W3) + b3
    probs = sigmoid(Z3)

    preds = (probs >= 0.5).astype(int)

    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds)
    rec = recall_score(y_test, preds)
    f1 = f1_score(y_test, preds)
    cm = confusion_matrix(y_test, preds)

    return loss_history, acc_history, acc, prec, rec, f1, cm


# ========================================================
# 4. TRAIN TEST SPLIT — 80/20
# ========================================================
print("\n=========== PROGRAM 1 — TRAIN/TEST = 80/20 ===========")

X_train, X_test, y_train, y_test = train_test_split(
    X_full, y_full, test_size=0.2, random_state=42, stratify=y_full
)

X_train_vec = vectorizer.transform(X_train).toarray()
X_test_vec = vectorizer.transform(X_test).toarray()

learning_rates = [0.001, 0.01, 0.005]

all_loss = {}
all_acc = {}

for lr in learning_rates:
```

```
    print(f"\n Learning Rate: {lr}")
    loss_hist, acc_hist, acc, prec, rec, f1, cm = train_model(
        X_train_vec, y_train, X_test_vec, y_test, lr
    )

    all_loss[lr] = loss_hist
    all_acc[lr] = acc_hist

    print("Accuracy :", acc)
    print("Precision:", prec)
    print("Recall   :", rec)
    print("F1 Score :", f1)
    print("Confusion Matrix:\n", cm)


# ========== Combined Loss Curve ==========
plt.figure(figsize=(8,5))
for lr in all_loss:
    plt.plot(all_loss[lr], label=f"LR={lr}")
plt.title("Combined Loss Curve (80/20 Split)")
plt.xlabel("Epochs")
plt.ylabel("BCE Loss")
plt.grid(True)
plt.legend()
plt.show()

# ========== Combined Accuracy Curve ==========
plt.figure(figsize=(8,5))
for lr in all_acc:
    plt.plot(all_acc[lr], label=f"LR={lr}")
plt.title("Combined Accuracy Curve (80/20 Split)")
plt.xlabel("Epochs")
plt.ylabel("Training Accuracy")
plt.grid(True)
plt.legend()
plt.show()
```

```
=========== PROGRAM 1 — TRAIN/TEST = 80/20 ===========

 Learning Rate: 0.001

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

Accuracy : 0.8333333333333334
Precision: 0.0
```

```
Recall   : 0.0
F1 Score : 0.0
Confusion Matrix:
 [[500   0]
 [100   0]]

 Learning Rate: 0.01

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

Accuracy : 0.8333333333333334
Precision: 0.0
Recall   : 0.0
F1 Score : 0.0
Confusion Matrix:
 [[500   0]
 [100   0]]

 Learning Rate: 0.005

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

Accuracy : 0.8333333333333334
Precision: 0.0
Recall   : 0.0
F1 Score : 0.0
Confusion Matrix:
 [[500   0]
 [100   0]]
```


Combined Loss Curve (80/20 Split)
BCE Loss
Epochs
0.70
0.68
0.66
0.64
0.62
0.60
0.58
0.56
0
25
50
75
100
125
150
175
200
LR=0.001
LR=0.01
LR=0.005


Combined Accuracy Curve (80/20 Split)
Training Accuracy
Epochs
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0
25
50
75
100
125
150
175
200
LR=0.001
LR=0.01
LR=0.005

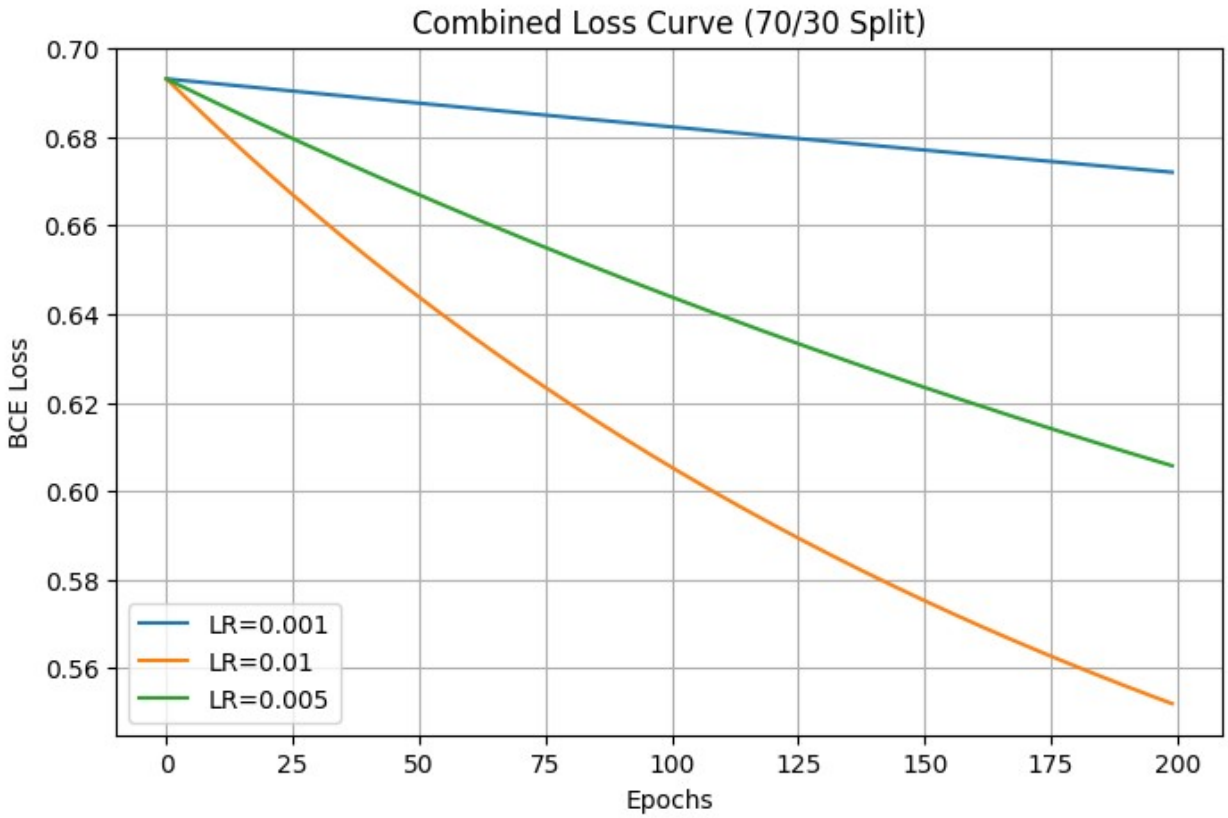
```python
# ============================
# PROGRAM 2 – TRAIN/TEST = 70/30
# ============================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix

df = pd.read_csv("/content/spam_or_not_spam.csv")
df = df[['email','label']].dropna().reset_index(drop=True)

X_full = df["email"]
y_full = df["label"].values.reshape(-1,1)

vectorizer = TfidfVectorizer(stop_words='english', max_features=4000,
ngram_range=(1,2))
vectorizer.fit(X_full)

def sigmoid(z): return 1/(1+np.exp(-z))
def relu(z): return np.maximum(0,z)
def relu_derivative(z): return (z>0).astype(float)
def bce_loss(y,y_pred): return -np.mean(y*np.log(y_pred+1e-8)+(1-
y)*np.log(1-y_pred+1e-8))

# same NN function copied exactly from program 1 here…

def train_model(X_train_vec, y_train, X_test_vec, y_test, lr):

    epochs=200
    m,n = X_train_vec.shape

    h1,h2 = 64,32

    W1=np.random.randn(n,h1)*0.01; b1=np.zeros((1,h1))
    W2=np.random.randn(h1,h2)*0.01; b2=np.zeros((1,h2))
    W3=np.random.randn(h2,1)*0.01; b3=np.zeros((1,1))

    loss_history=[]; acc_history=[]

    for epoch in range(epochs):

        Z1=np.dot(X_train_vec,W1)+b1
        A1=relu(Z1)

        Z2=np.dot(A1,W2)+b2
```

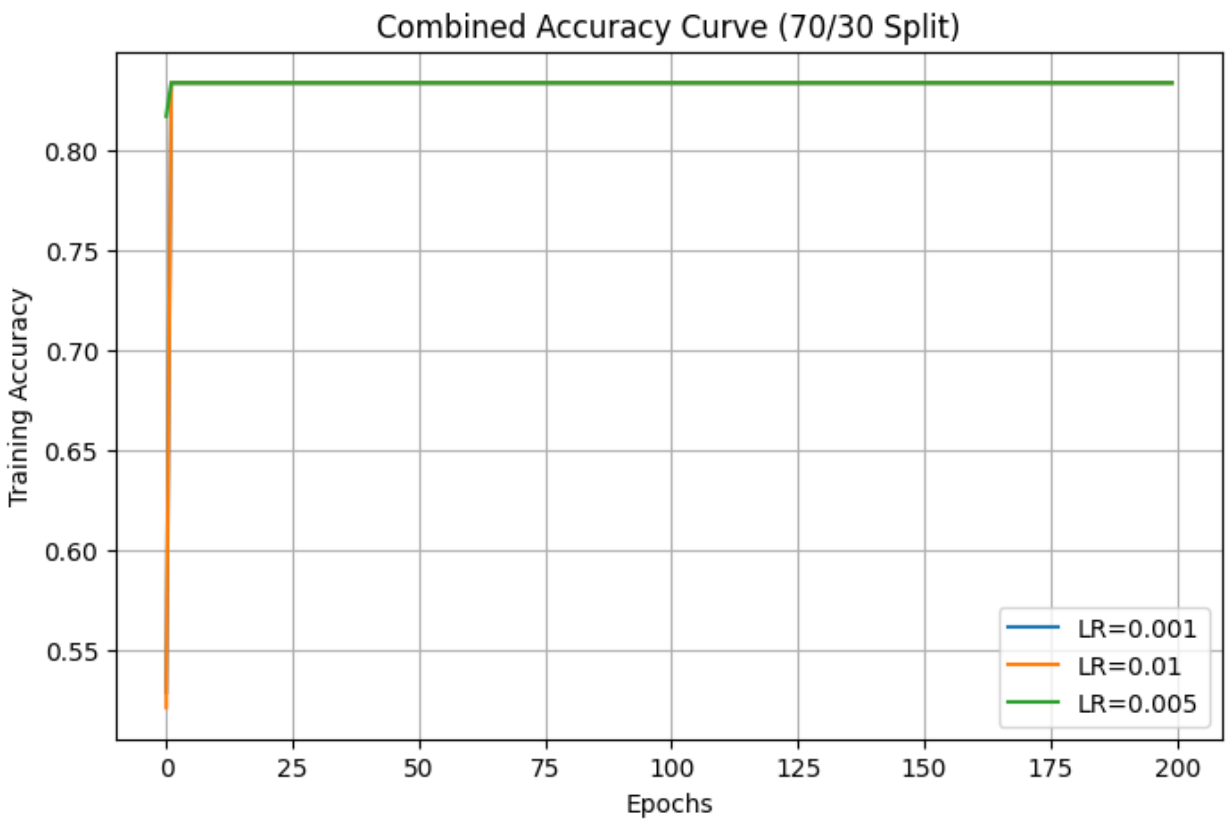
```
        A2=relu(Z2)

        Z3=np.dot(A2,W3)+b3
        A3=sigmoid(Z3)

        loss=bce_loss(y_train,A3)
        loss_history.append(loss)

        train_pred=(A3>=0.5).astype(int)
        acc=accuracy_score(y_train,train_pred)
        acc_history.append(acc)

        dZ3=A3-y_train
        dW3=np.dot(A2.T,dZ3)/m
        db3=np.mean(dZ3,keepdims=True)

        dA2=np.dot(dZ3,W3.T)
        dZ2=dA2*relu_derivative(Z2)
        dW2=np.dot(A1.T,dZ2)/m
        db2=np.mean(dZ2,keepdims=True)

        dA1=np.dot(dZ2,W2.T)
        dZ1=dA1*relu_derivative(Z1)
        dW1=np.dot(X_train_vec.T,dZ1)/m
        db1=np.mean(dZ1,keepdims=True)

        W1-=lr*dW1; b1-=lr*db1
        W2-=lr*dW2; b2-=lr*db2
        W3-=lr*dW3; b3-=lr*db3

    Z1=np.dot(X_test_vec,W1)+b1; A1=relu(Z1)
    Z2=np.dot(A1,W2)+b2;       A2=relu(Z2)
    Z3=np.dot(A2,W3)+b3;       probs=sigmoid(Z3)

    preds=(probs>=0.5).astype(int)

    acc=accuracy_score(y_test,preds)
    prec=precision_score(y_test,preds)
    rec=recall_score(y_test,preds)
    f1=f1_score(y_test,preds)
    cm=confusion_matrix(y_test,preds)

    return loss_history,acc_history,acc,prec,rec,f1,cm


print("\n=========== PROGRAM 2 — TRAIN/TEST = 70/30 ===========")

# 70/30 split
X_train, X_test, y_train, y_test = train_test_split(
    X_full, y_full, test_size=0.3, random_state=42, stratify=y_full
)
```

```
X_train_vec = vectorizer.transform(X_train).toarray()
X_test_vec = vectorizer.transform(X_test).toarray()

learning_rates = [0.001, 0.01, 0.005]

all_loss = {}
all_acc = {}

for lr in learning_rates:

    print(f"\n Learning Rate = {lr}")

    loss_hist, acc_hist, acc, prec, rec, f1, cm = train_model(
        X_train_vec, y_train, X_test_vec, y_test, lr
    )

    all_loss[lr] = loss_hist
    all_acc[lr] = acc_hist

    print("Accuracy :", acc)
    print("Precision:", prec)
    print("Recall   :", rec)
    print("F1 Score :", f1)
    print("Confusion Matrix:\n", cm)


# -------- Combined Loss Curve --------
plt.figure(figsize=(8,5))
for lr in all_loss:
    plt.plot(all_loss[lr], label=f"LR={lr}")
plt.title("Combined Loss Curve (70/30 Split)")
plt.xlabel("Epochs")
plt.ylabel("BCE Loss")
plt.grid(True)
plt.legend()
plt.show()

# -------- Combined Accuracy Curve --------
plt.figure(figsize=(8,5))
for lr in all_acc:
    plt.plot(all_acc[lr], label=f"LR={lr}")
plt.title("Combined Accuracy Curve (70/30 Split)")
plt.xlabel("Epochs")
plt.ylabel("Training Accuracy")
plt.grid(True)
plt.legend()
plt.show()
```

```
=========== PROGRAM 2 — TRAIN/TEST = 70/30 ===========
```

```
 Learning Rate = 0.001

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

Accuracy : 0.8333333333333334
Precision: 0.0
Recall   : 0.0
F1 Score : 0.0
Confusion Matrix:
 [[750   0]
 [150   0]]

 Learning Rate = 0.01

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

Accuracy : 0.8333333333333334
Precision: 0.0
Recall   : 0.0
F1 Score : 0.0
Confusion Matrix:
 [[750   0]
 [150   0]]

 Learning Rate = 0.005

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

Accuracy : 0.8333333333333334
Precision: 0.0
Recall   : 0.0
F1 Score : 0.0
Confusion Matrix:
 [[750   0]
 [150   0]]
```


Combined Loss Curve (70/30 Split)
BCE Loss
Epochs
0.70
0.68
0.66
0.64
0.62
0.60
0.58
0.56
0
25
50
75
100
125
150
175
200
LR=0.001
LR=0.01
LR=0.005


Combined Accuracy Curve (70/30 Split)
Training Accuracy
Epochs
0.80
0.75
0.70
0.65
0.60
0.55
0
25
50
75
100
125
150
175
200
LR=0.001
LR=0.01
LR=0.005